Pluto vs DaCe auto_optimize on the non-PolyBench NPBench kernels

Median speedup over NumPy (50 repetitions)

	Pluto	DaCe auto_opt	NumPy	
geo-mean	**1.8x**	**1.9x**	**315.4 ms**	
cholesky2	0.47x	0.66x	21.8 ms	PolyBench family - no Pluto reference yet
covariance	2.9x	0.45x	28.0 ms	PolyBench family - no Pluto reference yet
covariance2	0.58x	crashed	5.5 ms	PolyBench family - no Pluto reference yet
doitgen	0.15x	1.6x	238.6 ms	PolyBench family - no Pluto reference yet
gesummv	26.8x	5.4x	197.3 ms	PolyBench family - no Pluto reference yet
jacobi_1d	18.4x	0.69x	257.9 ms	PolyBench family - no Pluto reference yet
k2mm	0.09x	1.4x	167.8 ms	PolyBench family - no Pluto reference yet
k3mm	0.06x	1.0x	177.7 ms	PolyBench family - no Pluto reference yet
symm	7.1x	0.61x	7.27 s	PolyBench family - no Pluto reference yet
conv2d_bias	declined	0.23x	7.59 s	deep learning
lenet	declined	0.40x	1.46 s	deep learning
mlp	0.02x	0.76x	23.7 ms	deep learning
resnet	0.34x	0.34x	1.54 s	deep learning
softmax	1.0x	6.3x	831.3 ms	deep learning
hdiff	6.4x	13.7x	152.6 ms	weather stencils
vadv	11.4x	invalid	544.4 ms	weather stencils
cavity_flow	1.7x	crashed	1.61 s	physics / CFD
channel_flow	2.9x	0.88x	2.86 s	physics / CFD
contour_integral	0.04x	2.2x	492.0 ms	physics / CFD
nbody	1.5x	crashed	297.8 ms	physics / CFD
scattering_self_energies	declined	0.17x	1.06 s	physics / CFD
arc_distance	59.2x	12.3x	463.1 ms	signal / integration
azimint_hist	declined	3.9x	14.8 ms	signal / integration
azimint_naive	5.3x	11.2x	491.3 ms	signal / integration
compute	4.8x	6.4x	436.1 ms	signal / integration
stockham_fft	declined	0.01x	95.5 ms	signal / integration
crc16	168x	113x	1.41 s	misc / non-affine
go_fast	1.4x	1.7x	108.6 ms	misc / non-affine
mandelbrot1	11.6x	13.3x	877.8 ms	misc / non-affine
mandelbrot2	declined	0.61x	365.5 ms	misc / non-affine
spmv	declined	494x	237.8 ms	misc / non-affine

0.031x 0.1x 0.5x 1x 2x 10x 32x

Speedup over NumPy (log scale). Hatched grey: no valid measurement, see page 2.

Pluto vs DaCe auto_optimize on the non-PolyBench NPBench kernels: detailed results

Preset paper, 50 repetitions per kernel. Every blank cell on page 1 is accounted for below; a speedup is given only where the result validated.

DaCe column: dace_cpu_autoopt / extended@eb7b1352a

Pluto: polycc --tile --parallel --codegen-context=1 (Clan frontend), clang -O3 -march=native

kernel	NumPy runtime	Pluto	DaCe auto_opt	validated	note / reason a cell is blank
cholesky2	21.8 ms	0.47x	0.66x	yes	both columns validated against the NumPy reference
covariance	28.0 ms	2.9x	0.45x	yes	both columns validated against the NumPy reference
covariance2	5.5 ms	0.58x	crashed	no	DaCe: ModuleNotFoundError: No module named 'npbench.benchmarks.polybench.covariance2.covariance2_dace'
doitgen	238.6 ms	0.15x	1.6x	yes	both columns validated against the NumPy reference
gesummv	197.3 ms	26.8x	5.4x	yes	both columns validated against the NumPy reference
jacobi_1d	257.9 ms	18.4x	0.69x	yes	both columns validated against the NumPy reference
k2mm	167.8 ms	0.09x	1.4x	yes	both columns validated against the NumPy reference
k3mm	177.7 ms	0.06x	1.0x	yes	both columns validated against the NumPy reference
symm	7.27 s	7.1x	0.61x	yes	Pluto: correct and tiled, but polycc marked no loop parallel -- this is a SINGLE-THREADED result. The kernel is an inherently sequential recurrence and both polycc frontends agree there is no parallelism to find, so the speedup is not comparable to the 72-thread rows above.
conv2d_bias	7.59 s	declined	0.23x	DaCe only	Pluto: Pluto transformation: polycc did not terminate within its timeout.
lenet	1.46 s	declined	0.40x	DaCe only	Pluto: Pluto transformation: polycc did not terminate within its timeout.
mlp	23.7 ms	0.02x	0.76x	yes	both columns validated against the NumPy reference
resnet	1.54 s	0.34x	0.34x	yes	both columns validated against the NumPy reference
softmax	831.3 ms	1.0x	6.3x	yes	both columns validated against the NumPy reference
hdiff	152.6 ms	6.4x	13.7x	yes	both columns validated against the NumPy reference
vadv	544.4 ms	11.4x	invalid	no	DaCe: ran but did not validate against the NumPy reference
cavity_flow	1.61 s	1.7x	crashed	no	DaCe: no result row and no recognised failure in the log
channel_flow	2.86 s	2.9x	0.88x	yes	both columns validated against the NumPy reference
contour_integral	492.0 ms	0.04x	2.2x	yes	both columns validated against the NumPy reference
nbody	297.8 ms	1.5x	crashed	no	DaCe: encountered in File "/users/levwidmer/npbench/npbench/benchmarks/nbody/nbody_dace.py", line 56, column 9
scattering_self_energies	1.06 s	declined	0.17x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.
arc_distance	463.1 ms	59.2x	12.3x	yes	both columns validated against the NumPy reference
azimint_hist	14.8 ms	declined	3.9x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.
azimint_naive	491.3 ms	5.3x	11.2x	yes	both columns validated against the NumPy reference
compute	436.1 ms	4.8x	6.4x	yes	both columns validated against the NumPy reference
stockham_fft	95.5 ms	declined	0.01x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.
crc16	1.41 s	168x	113x	yes	Pluto: correct and tiled, but polycc marked no loop parallel -- this is a SINGLE-THREADED result. The kernel is an inherently sequential recurrence and both polycc frontends agree there is no parallelism to find, so the speedup is not comparable to the 72-thread rows above.
go_fast	108.6 ms	1.4x	1.7x	yes	both columns validated against the NumPy reference
mandelbrot1	877.8 ms	11.6x	13.3x	yes	both columns validated against the NumPy reference
mandelbrot2	365.5 ms	declined	0.61x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.
spmv	237.8 ms	declined	494x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.